

Build Plan: Explainable Chest X-Ray Report Generation with Built-in Hallucination Detection

How to Use This Plan

This plan is written as a sequence of phases. Each phase has numbered steps, a **checkpoint** (things to verify before moving on), and a **"If you're not sure" note** flagging where teams commonly rush and make mistakes. Do not skip a checkpoint. If a checkpoint fails, stop and fix it before continuing — problems compound fast in ML pipelines, and a bug in Phase 2 will silently corrupt everything built on top of it in Phase 7.

Work through phases in order. Phases 3–4 (classifier) and Phase 5 (retrieval corpus) can be done in parallel by different team members since they don't depend on each other. Everything from Phase 6 onward depends on Phases 3–5 being complete.

Team of 4 suggested split:

- **Member A:** Phases 0–2 (setup, data), then Phase 9 (evaluation)
- **Member B:** Phases 3–4 (classifier + Grad-CAM)
- **Member C:** Phase 5 (retrieval/RAG) + Phase 6 (patient history)
- **Member D:** Phase 7 (medical LLM integration) + Phase 10 (demo)
- Phase 8 (hallucination detection) and Phase 11 (integration) should be done together as a team, since it touches everyone's code.

Phase 0 — Environment Setup

Goal: everyone on the team has an identical, working environment before anyone writes model code.

1. Create a shared GitHub repository. Add a `.gitignore` that excludes `*.pt`, `*.pth`, `data/`, `__pycache__`, and `.env`.
2. Decide on ONE compute environment for training (Google Colab Pro or Kaggle Notebooks recommended — both give free/cheap GPU access). Everyone should develop against the same Python version to avoid "works on my machine" issues.
3. Create a Python virtual environment:

```
bash
```

```
python3 -m venv venv
source venv/bin/activate # or venv\Scripts\activate on Windows
```

4. Install core dependencies:

```
bash
```

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org
pip install transformers accelerate sentence-transformers faiss-cpu chromadb
pip install scikit-learn pandas numpy matplotlib seaborn
pip install nltk rouge-score pycocoevalcap
pip install grad-cam opencv-python pillow
pip install streamlit flask
pip install jupyter
```

5. Freeze the environment immediately: `(pip freeze > requirements.txt)` and commit it. This is what saves you when someone's code "randomly" stops working two weeks later because a library updated.
6. Set up a shared folder structure in the repo:

```
project/
  data/                # raw + processed dataset (gitignored)
  notebooks/           # exploration notebooks
  src/
    data/              # dataset loading + preprocessing code
    classifier/        # CNN findings classifier
    retrieval/         # vector DB + RAG pipeline
    history/           # patient history handling
    generation/        # medical LLM report generation
    hallucination/     # hallucination detection module
    explainability/    # Grad-CAM code
    eval/              # evaluation scripts
  demo/               # Streamlit/Flask app
  checkpoints/        # saved model weights (gitignored)
  docs/               # your report, diagrams, proposal PDF
```

Checkpoint: Every team member can run `(import torch; print(torch.cuda.is_available()))` and get `(True)` on the shared compute environment. If anyone gets `(False)`, fix this before writing any training code — training on CPU will silently work but take 20–50x longer, and you'll waste days before realizing why things are slow.

If you're not sure: whether your Colab session has a GPU attached — check Runtime → Change runtime type → Hardware accelerator = GPU, every single session. Colab resets this sometimes.

Phase 1 — Data Acquisition & Understanding

Goal: get the IU Chest X-Ray dataset downloaded, and actually look at it before writing a single line of model code.

1. Download the IU Chest X-Ray dataset (Open-i, Indiana University). It is typically distributed as:
 - A set of PNG/DICOM chest X-ray images
 - An XML or CSV file per case containing the report text (Findings + Impression sections) and MeSH-based finding labels
2. Search "Indiana University Chest X-ray Open-i dataset download" or check Kaggle for a pre-packaged mirror (search "IU X-ray Kaggle") — this is usually easier than dealing with raw Open-i XML exports.
3. Once downloaded, DO NOT immediately write code. First, manually open 10-15 random image+report pairs and read them. You are checking:
 - Are images frontal, lateral, or both? (You likely want frontal only, or need to handle both.)
 - Are report sections consistently labeled (Findings vs. Impression)?
 - Are there empty/missing reports?
 - What does a "No Finding" report look like vs. one with an actual finding?
4. Write a small exploration notebook (`notebooks/01_explore_data.ipynb`) that:
 - Loads the metadata into a pandas DataFrame
 - Counts total image-report pairs
 - Plots the distribution of finding labels (expect heavy imbalance — "No Finding" or "Normal" will dominate)
 - Displays 5 sample images with their reports side by side
5. Note the exact total count of usable pairs after removing broken/missing entries. Write this number down — you will need it to sanity-check your train/val/test split sizes later.

Checkpoint: You have a CSV/dataframe with columns like `image_path`, `report_text`, `findings_labels` and you have personally read at least 15 real reports. You know the class imbalance ratio (e.g., "70% No Finding, 30% split across 13 other conditions").

If you're not sure: whether a "Findings" vs. "Impression" section split matters — it does. "Findings" is the detailed description; "Impression" is the short clinical summary. Decide explicitly which one (or both, concatenated) your model will generate, and use the SAME choice consistently across the classifier labels, the generation target, and evaluation. Mixing this up later is a very common and hard-to-detect bug.

Phase 2 — Data Preprocessing

Goal: turn raw images + text into clean, split, model-ready data.

1. Image preprocessing:

- Resize all images to a fixed size (224×224 is standard for DenseNet121).
- Convert grayscale X-rays to 3-channel if your pretrained backbone expects RGB (duplicate the single channel 3x).
- Normalize using ImageNet mean/std if using ImageNet-pretrained weights:
`mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`.

2. Text preprocessing:

- Lowercase report text, strip extra whitespace.
- Do NOT aggressively remove medical terms/punctuation that carry clinical meaning (e.g., don't strip "-" from "non-productive").
- Tokenize using the tokenizer that matches whatever decoder/LLM you'll use later (keep this consistent — decide this now, not in Phase 7).

3. Label preprocessing:

- Fix your finding label vocabulary now (e.g., the standard 14 CheXpert-style labels: Cardiomegaly, Edema, Consolidation, Pneumonia, Atelectasis, Pneumothorax, Pleural Effusion, No Finding, etc.). Write this list down in a config file — every module (classifier, hallucination checker) must reference the SAME fixed label list.
- Convert to multi-hot vectors (one 0/1 per label per case).

4. Train/Val/Test split:

- Split at the PATIENT level if patient IDs are available (not just at the image level) — otherwise the same patient's images could leak across train and test, inflating your reported accuracy artificially.
- Suggested split: 80% train / 10% validation / 10% test.
- Save the split as fixed lists of IDs (`train_ids.txt`, `val_ids.txt`, `test_ids.txt`) so every team member uses the exact same split.

5. Build a PyTorch `Dataset` class that returns `(image_tensor, report_text, label_vector, patient_id)` for a given index. Test it by loading a single batch and printing shapes.

Checkpoint: Loading a batch of size 8 gives you an image tensor of shape `[8, 3, 224, 224]`, a label tensor of shape `[8, 14]` (or however many labels), and 8 report strings. Print and manually read 2 of them to confirm they match their images.

If you're not sure: whether you have data leakage between splits — check this explicitly with code: `assert len(set(train_ids) & set(test_ids)) == 0`. Run this assertion every time you touch the split logic.

Phase 3 — Findings Classifier (CNN)

Goal: a trained multi-label classifier that predicts findings from an X-ray image. This is the "source of truth" the rest of your pipeline cross-checks against, so get this right before moving on.

1. Load a pretrained DenseNet121

```
((torchvision.models.densenet121(weights='IMAGENET1K_V1'))).
```

2. Replace the final classification layer with a new `Linear` layer with output size = number of findings (e.g., 14), followed by a sigmoid (since this is multi-label, NOT multi-class — a case can have multiple findings, so use `BCEWithLogitsLoss`, not `CrossEntropyLoss`)).
3. Freeze the early convolutional blocks initially; only fine-tune the last dense block + your new classification head. This trains faster and avoids overfitting on a small dataset.
4. Handle class imbalance:
 - Compute per-class positive weights: `pos_weight = (num_negative / num_positive)` per class, and pass this to `BCEWithLogitsLoss(pos_weight=...)`.
 - Without this, your model will likely just learn to predict "No Finding" for everything and still get deceptively high raw accuracy.
5. Training loop essentials:
 - Optimizer: AdamW, learning rate $\sim 1e-4$ for the head, $\sim 1e-5$ if fine-tuning backbone layers.
 - Use a learning rate scheduler (e.g., `ReduceLROnPlateau`) on validation loss).
 - Track per-class AUC-ROC on the validation set after every epoch, not just loss.
 - Save the model checkpoint with the BEST validation AUC, not just the last epoch.
6. Train for an initial run of $\sim 15-20$ epochs. Plot training vs. validation loss curves. If validation loss starts rising while training loss keeps dropping, you're overfitting — add dropout, more augmentation, or stop earlier.
7. On the held-out test set, report:
 - Per-class AUC-ROC
 - Per-class F1 at a chosen threshold (start with 0.5, but also try tuning per-class thresholds using the validation set)
 - A confusion-style summary (or per-class precision/recall table)

Checkpoint: You have a saved checkpoint file, and per-class AUC scores that are meaningfully above 0.5 (random) for at least the more common findings. If AUC is close to 0.5 for everything, something is wrong (label mismatch, bad normalization, or learning rate too high/low) — do not proceed to Grad-CAM or generation until this is fixed.

If you're not sure: whether your AUC numbers are "good enough" — compare them to numbers reported in the CheXNet/CheXpert papers (Section 12 references) for context. You will not beat those numbers with a much smaller dataset and less compute, and that's fine — the goal is a reasonably working classifier, not state-of-the-art performance. Say this explicitly in your report rather than overclaiming.

Phase 4 — Explainability (Grad-CAM)

Goal: visualize which image region drove each predicted finding.

1. Use the `(pytorch-grad-cam)` library or implement Grad-CAM manually by hooking the last convolutional layer of your trained DenseNet121.
2. For a given image and a given predicted finding (class index), compute the Grad-CAM heatmap and overlay it on the original X-ray using `(cv2.addWeighted)`.
3. Test this on 5–10 known cases where you already know the ground truth finding location (from the radiologist's report, if it mentions laterality/location) — sanity check that the heatmap roughly makes sense (e.g., a "Cardiomegaly" heatmap should highlight around the heart, not the edge of the image).
4. Save this as a reusable function: `(generate_gradcam(model, image_tensor, target_class_index) -> heatmap_overlay_image)`.

Checkpoint: You can pass any test-set image + finding index into your function and get back a heatmap that visually overlaps with a clinically plausible region for that finding.

If you're not sure: whether your Grad-CAM implementation is hooking the correct layer — if the heatmap looks like random noise or highlights only image borders, you likely picked too early or too late a layer. For DenseNet121, hook `(model.features.denseblock4)` (the last dense block) or `(model.features.norm5)`, and try both if one doesn't look right.

Phase 5 — Retrieval Corpus & Vector Database (RAG)

Goal: build the retrieval system that supplies relevant prior cases and guideline text to the report generator.

1. **Build the corpus** (two sources, combine both):
 - **Prior cases:** use a subset of your OWN training set reports (not test set — this would be data leakage) as "prior case" documents to retrieve from.
 - **Guidelines:** collect a small set of publicly available radiology reference material (e.g., open-access radiology textbook chapters, ACR Appropriateness Criteria summaries, or Radiopaedia-style condition descriptions for the findings in your label list). Keep this corpus small and curated (30–100 documents is enough for a capstone) — quality over quantity.

2. **Chunk the documents** into passages of roughly 100–300 words each. Store each chunk with metadata (source, finding it relates to).
3. **Generate embeddings** for every chunk using a sentence embedding model:

python

```
from sentence_transformers import SentenceTransformer
embedder = SentenceTransformer('all-MiniLM-L6-v2') # or a BioBERT-based emb
embeddings = embedder.encode(chunks)
```

4. **Build the vector index** using FAISS:

python

```
import faiss
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)
```

(ChromaDB is an easier drop-in alternative if you want built-in persistence and metadata filtering without managing FAISS indices manually.) 5. **Write the retrieval function:** given a query (e.g., the classifier's top predicted findings turned into a short text query, like "cardiomegaly pleural effusion"), embed the query and retrieve the top-3 nearest chunks. 6. **Manually evaluate retrieval quality before wiring it into anything else.** Run 10 sample queries and read the top-3 results yourself. Are they actually relevant? If retrieval quality is poor, the generation step downstream will be poor no matter how good your LLM is — garbage in, garbage out.

Checkpoint: For at least 8 out of 10 manually tested queries, the top-3 retrieved chunks are clinically relevant to the query.

If you're not sure: what to use as the retrieval query since you don't have the report yet at inference time — use the findings classifier's predicted labels (from Phase 3) as the query. This is exactly why the classifier needs to run BEFORE retrieval in your pipeline, not after.

Phase 6 — Patient History Module

Goal: represent patient history as structured text the generation model can use, and set up the data needed to test contradiction detection later.

1. Since the IU X-Ray dataset has no real structured EHR data, extract what you can from the "Comparison" field in the original reports (many mention things like "compared to prior exam dated..."), and construct a simple structured schema:

json

```
{
  "age": 54,
  "sex": "F",
  "previous_reports": ["Prior CXR (2 years ago): mild cardiomegaly, no effus
  "lab_values": {"WBC": "12.4", "CRP": "elevated"}
}
```

2. Where real prior-report data isn't available for a given case, honestly document that you are constructing **synthetic/simulated patient history** for the purpose of demonstrating and testing the clinical-inconsistency-checking module. This is a normal and defensible approach for a capstone — just don't misrepresent it as real EHR data in your report.
3. Deliberately construct a small set (20–30 cases) of **synthetic contradiction test cases** — e.g., a history mentioning "cast removed 2 years ago, fracture healed" paired with a report that (for testing purposes) says "new fracture observed." You need these deliberately-constructed contradictions to actually test whether your Phase 8 hallucination checker works, since real contradictions will be rare and hard to find on demand.
4. Write a function that formats a patient history dict into a clean text string to prepend/append to the LLM prompt in Phase 7.

Checkpoint: You have a JSON file of patient histories for your test set (real where derivable, clearly-labeled synthetic otherwise), plus a separate small labeled set of deliberate contradiction test cases for validating Phase 8.

If you're not sure: how much synthetic data is "too much" — as long as your report is explicit and honest about what's real vs. simulated, and you're not claiming clinical validity, this is fine. Transparency here is a strength in your viva, not a weakness.

Phase 7 — Medical LLM Integration (Report Generation)

Goal: generate a free-text report from the image, retrieved context, and patient history.

1. **Choose one model** (don't try to integrate all three):
 - **LLaVA-Med** is the most accessible open-source option with available weights and code — recommended starting point.
 - Med-PaLM 2 is not openly available for fine-tuning/self-hosting (Google-internal access), so if you pick this, you're realistically describing it as a comparative reference in your literature review rather than something you run yourself.
 - Qwen-VL(-Medical) is another viable open option if LLaVA-Med proves difficult to set up.

- **Decide this in week 1 of this phase and don't switch later** — switching models mid-way wastes enormous time re-doing prompt formatting and integration work.
2. Given compute constraints, plan to use the model via **frozen inference with prompt engineering** first (no fine-tuning). Only attempt lightweight LoRA fine-tuning if you have time left after a working baseline exists.
 3. **Construct the prompt** by combining:
 - The X-ray image (passed as the model expects — check the specific model's input format)
 - The retrieved top-3 chunks from Phase 5, formatted as: `"Relevant prior cases and guidelines:\n[chunk1]\n[chunk2]\n[chunk3]"`
 - The patient history text from Phase 6: `"Patient history: age 54, female, prior CXR 2 years ago showed mild cardiomegaly..."`
 - A clear instruction: `"Based on the image, the above prior cases/guidelines, and the patient's history, write a radiology report (Findings and Impression)."`
 4. Run inference on a handful of test images first and manually read the outputs. Are they at all reasonable, or garbled/off-topic? Debug the prompt format before scaling up.
 5. Set up batch inference to generate reports for your full test set, saving `(image_id, generated_report)` pairs.

Checkpoint: You can generate a coherent (even if imperfect) report for any test-set image, and you have manually read at least 15 generated reports to get a feel for typical failure patterns (e.g., does it hallucinate confidently? Is it too generic? Does it actually use the retrieved context, or ignore it?).

If you're not sure: whether the model is actually using the retrieved context or just ignoring it — test this directly: run the same image with the retrieved context included vs. deliberately swapped for irrelevant/wrong context, and check whether the output changes. If it doesn't change at all, your prompt integration isn't working and needs debugging before you proceed.

Phase 8 — Hallucination Detection Module

Goal: the core novelty of your project — flag two categories of unsupported claims.

1. **Findings extraction from generated text:** write a function that scans the generated report text for mentions of any label in your fixed finding vocabulary (from Phase 2), using keyword/synonym matching (e.g., "enlarged heart" → Cardiomegaly; "fluid in the pleural space" → Pleural Effusion). Build a small synonym dictionary per finding — this is manual work but essential and worth doing carefully.

2. Check 1 — Image-unsupported hallucination:

- Compare the findings extracted from the report against the classifier's predicted findings (Phase 3) for the same image (using a fixed probability threshold, e.g., >0.5).
- Any finding mentioned in the report but NOT predicted by the classifier → flag as `image_unsupported_hallucination`.

3. Check 2 — Clinical inconsistency hallucination:

- Compare findings/statements extracted from the report against the patient history (Phase 6).
- Start with simple, explicit rule patterns (not a complex NLI model, at least initially): e.g., if the report says "new [X]" and the history text contains "[X]" with words like "resolved," "removed," "healed," or "prior" nearby, flag as `clinical_inconsistency_hallucination`.
- Test this specifically against your deliberately-constructed contradiction cases from Phase 6, step 3.

4. Output format: for each generated report, produce a structured result:

```
json
{
  "report": "...",
  "predicted_findings": ["Cardiomegaly"],
  "report_mentioned_findings": ["Cardiomegaly", "Pneumonia"],
  "image_unsupported_hallucinations": ["Pneumonia"],
  "clinical_inconsistency_hallucinations": []
}
```

- ## 5. Validate manually:
- take 20 generated reports, manually read them alongside the flags your system produced, and judge whether each flag is correct. Compute your own rough precision (of the flags raised, how many were actually correct?) and recall (of the actual problems you can find by reading, how many did the system catch?).

Checkpoint: On your 20–30 deliberate contradiction test cases (Phase 6), Check 2 catches the large majority of them. On your general test set, Check 1's flags are manually verified as sensible on at least 15/20 spot-checked cases.

If you're not sure: how strict to make the flagging (too strict = many false alarms, too loose = misses real issues) — tune this using your validation set, not your test set, and report the trade-off honestly (this is a great thing to discuss in your viva — every detection system has a precision/recall trade-off, and showing you understand it is a strong signal).

Phase 9 — Evaluation

Goal: produce the final numbers for your report.

1. **Classifier:** per-class AUC-ROC, per-class F1 (already done in Phase 3, finalize on test set).
2. **Report generation quality:** compute BLEU and ROUGE scores between generated reports and ground-truth reports on the test set using `pycocoevalcap` or `rouge-score`.
3. **Hallucination detection:** report precision/recall for both hallucination types against your manually-labeled validation subset (you cannot get "true" labels for the full test set without manual review, so be upfront that this metric is measured on a manually reviewed sample, not the full test set — that's normal and expected for this kind of evaluation).
4. **Ablation (strongly recommended, easy to add):** compare report quality (BLEU/ROUGE) WITH vs. WITHOUT the retrieved RAG context, and WITH vs. WITHOUT patient history. This directly demonstrates whether your enhancements actually helped — a very strong thing to show in your viva.
5. Compile all results into a single results table/notebook that your final report will reference.

Checkpoint: You have one notebook or script that reproduces every number in your final report, end to end, from saved checkpoints/data — not numbers copy-pasted from scattered experiments you can no longer reproduce.

If you're not sure: whether your numbers are "good" — always report them alongside a brief honest note on limitations (small dataset, simulated patient history, etc.) rather than overclaiming. Panels respect this far more than inflated claims.

Phase 10 — Demo Interface

Goal: a working, presentable demo for your viva.

1. Build a simple Streamlit app (`demo/app.py`) with:
 - An image upload/selection widget (let users pick from a few pre-loaded test images)
 - A text area to enter/edit patient history
 - A "Generate Report" button
 - Display: generated report, predicted findings, Grad-CAM overlay, and any flagged hallucinations (highlighted clearly, e.g., in red text)
2. Pre-load 3–5 good example cases, including at least one deliberately constructed to trigger each hallucination type, so your demo reliably shows the feature working live without relying on luck.

3. Test the full demo flow at least 10 times before your presentation, including on the actual machine/network you'll present from.

Checkpoint: The demo runs end-to-end without crashing, on a machine other than the one it was built on (test this — dependency issues are extremely common).

If you're not sure: how much processing time is acceptable live during a demo — if any step takes more than 5–10 seconds, cache the result ahead of time for your pre-loaded examples so your live demo doesn't stall awkwardly in front of the panel.

Phase 11 — Integration Testing

Goal: make sure everything works together, not just in isolation.

1. Run the FULL pipeline (image → classifier → retrieval → LLM generation → dual hallucination check → Grad-CAM) on the entire test set in one script, start to finish, with no manual intervention.
2. Time how long this takes per image — if it's very slow, decide now whether that's acceptable for your demo or needs optimization (e.g., pre-computing embeddings, caching retrieval results).
3. Check for silent failures: log every case where any module returns an empty/error result, and manually review a sample of these.

Checkpoint: The full pipeline runs on 100% of your test set without crashing, and you have a log of any warnings/edge cases to mention honestly in your report.

Phase 12 — Documentation & Final Submission

1. Update your project report with final architecture diagrams, all evaluation numbers, and the ablation results.
 2. Include a clearly labeled "Limitations" section (small dataset, simulated patient history, rule-based hallucination matching rather than a trained NLI model, etc.) — this strengthens rather than weakens your submission.
 3. Prepare your viva materials: the live demo, 2–3 slides on architecture, and answers to the 25 Q&A prepared earlier in this project (now updated to reflect the RAG + patient history enhancements).
 4. Do a full dry run of your presentation and demo with the whole team at least once before the actual defense.
-

Appendix A — Common Mistakes Checklist (re-check before final submission)

- ☐ No data leakage between train/val/test splits (checked at patient level, not just image level)

- ☐ Classifier uses `BCEWithLogitsLoss` with `pos_weight`, not plain cross-entropy
- ☐ The same fixed finding-label vocabulary is used consistently across classifier, hallucination checker, and evaluation
- ☐ Retrieval corpus does not include test-set reports (only train-set + external guidelines)
- ☐ Grad-CAM is hooked to a sensible convolutional layer and manually sanity-checked
- ☐ Patient history data is clearly labeled as real vs. simulated in your report
- ☐ Hallucination detection precision/recall is reported on a manually reviewed sample, with that caveat stated explicitly
- ☐ The full pipeline has been run end-to-end at least once without manual intervention
- ☐ `requirements.txt` is up to date and the demo has been tested on a second machine

Appendix B — If You Get Stuck

For each phase, before asking for help, check:

1. Did you print/visualize intermediate outputs, or are you debugging blind?
2. Does the bug reproduce on a single, small, fixed example you can inspect by hand?
3. Have you checked shapes/types at every step (`print(tensor.shape)`, `print(type(x))`)? Shape mismatches are the single most common source of silent bugs in this kind of pipeline.
4. Is this a data problem, a model problem, or an integration problem? Isolate which one before trying to fix anything — most "the model isn't learning" issues turn out to be data/label bugs, not model architecture problems.